

Differentiable GPU-Parallelized Task and Motion Planning

William Shen^{1,2*}, Caelan Garrett², Nishanth Kumar^{1,2}, Ankit Goyal², Tucker Hermans^{2,3},
Leslie Pack Kaelbling¹, Tomás Lozano-Pérez¹, Fabio Ramos^{2,4}

¹MIT CSAIL, ²NVIDIA Research, ³University of Utah, ⁴University of Sydney

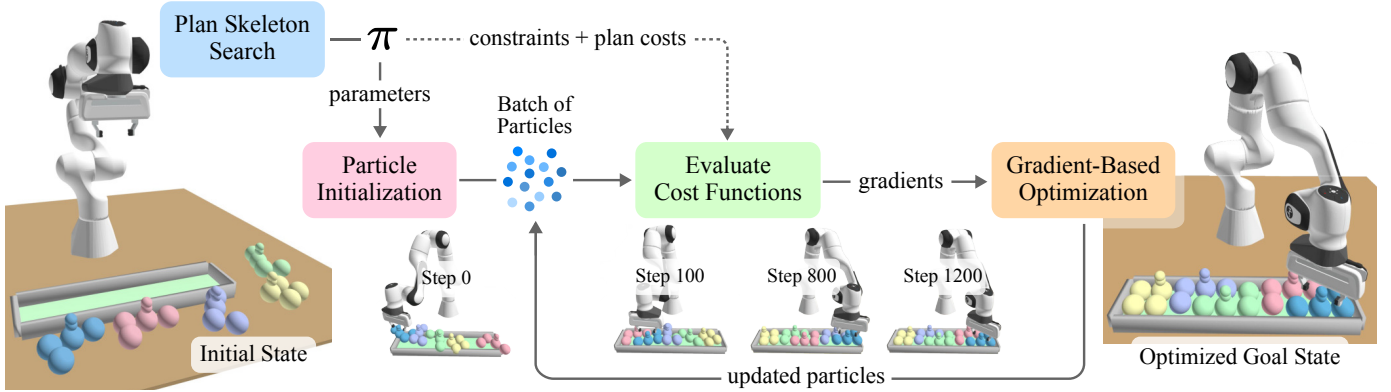


Fig. 1: **cuTAMP Overview.** cuTAMP frames TAMP as a backtracking bilevel search over plan skeletons (Sec. IV). Each skeleton π induces a continuous Constraint Satisfaction Problem that defines the structure of a particle (parameters) and cost functions (constraints and plan costs). These particles are optimized in parallel by evaluating their costs with differentiable cost functions (Eq. 4), allowing gradient-based optimizers to iteratively update them towards satisfying solutions (Sec. V-B).

Abstract—Planning long-horizon robot manipulation requires making discrete decisions about which objects to interact with and continuous decisions about how to interact with them. A robot planner must select grasps, placements, and motions that are feasible and safe. This class of problems falls under Task and Motion Planning (TAMP) and poses significant computational challenges in terms of algorithm runtime and solution quality, particularly when the solution space is highly constrained. To address these challenges, we propose a new bilevel TAMP algorithm that leverages GPU parallelism to efficiently explore thousands of candidate continuous solutions simultaneously. Our approach uses GPU parallelism to sample an initial batch of solution seeds for a plan skeleton and to apply differentiable optimization on this batch to satisfy plan constraints and minimize solution cost with respect to soft objectives. We demonstrate that our algorithm can effectively solve highly constrained problems with non-convex constraints in just seconds, substantially outperforming serial TAMP approaches, and validate our approach on multiple real-world robots. Project website and code: [cutamp.github.io](https://github.com/cutamp)

I. INTRODUCTION

Task and Motion Planning (TAMP) enables robots to plan long-horizon manipulation through integrated reasoning about sequences of discrete action types, such as pick, place, or press, and continuous action parameter values, such as grasps, placements, and trajectories [1]. TAMP planners have demonstrated remarkable generality in complex tasks including object rearrangement [2], multi-arm assembly [3], and cooking

a meal [4]. However, TAMP problems become increasingly challenging to solve efficiently as the horizon and action space increase, and the size of the set of solutions decreases due to tightly interacting constraints, e.g., kinematics and collisions.

A popular family of TAMP algorithms solve problems by first searching over discrete action sequences, also known as *plan skeletons*, and then searching for continuous action parameter values that satisfy the collective action *constraints* that govern legal parameter values. Each candidate plan skeleton induces a continuous *Constraint Satisfaction Problem* (CSP), which TAMP algorithms typically solve using a mixture of compositional *sampling* and joint *optimization* techniques, with each having their own trade-offs [1].

Sampling-based approaches to TAMP disconnect the parameters by generating samples for each independently using hand-engineered [5, 6, 7], projection-based [8], or learned generators [9, 10], and then combining them through composition and rejection. Because the parameters only interact through rejection sampling when evaluating constraints, many samples are often needed to satisfy problems where the constraints interact, such as tight packing problems (Figure 1). Optimization-based TAMP approaches, on the other hand, represent constraints as analytic functions in a mathematical program and solve for the continuous parameters by applying first- or second-order gradient descent [1, 11]. However, these constrained mathematical programs are highly non-convex with many local optima, making it challenging to find even a feasible solution from random parameter initializations.

*Work partially conducted during internship at NVIDIA. Correspondence to: William Shen (willshen@mit.edu), Caelan Garrett (cgarrett@nvidia.com).

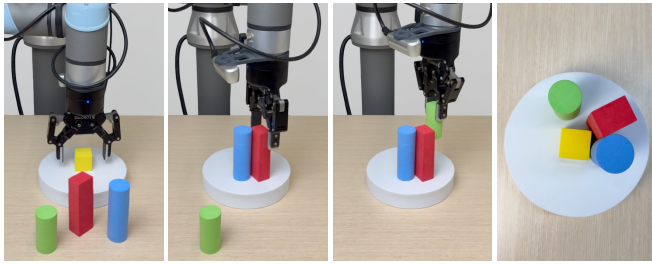


Fig. 3: **Object Packing with a UR5.** The objective is to place all objects onto the white region while minimizing the distance between them. The final state achieves a tight packing with successful reduction of the goal cost.

We present cuTAMP, the first GPU-parallelized TAMP planner. cuTAMP enables massively parallel exploration of TAMP solutions by combining ideas from sampling-based and optimization-based TAMP with GPU acceleration, going beyond prior serial algorithms. We treat TAMP constraint satisfaction as simultaneous differentiable optimization over a batch of *particles*, representing thousands of candidate solutions. This allows us to maintain the interdependence between continuous parameters by jointly optimizing them. To initialize the particles, we leverage parallelized samplers that solve *constraint subgraphs*, composing their generations to populate particles near the solution manifold while ensuring good coverage of parameter space. We demonstrate that when massively parallelized, cuTAMP can effectively solve highly constrained TAMP problems. Our approach inherits the locality of gradient descent and explores multiple basins through compositional sampling, increasing the likelihood of finding the global optima. Although we focus on GPU acceleration, our method applies to other forms of parallel computation.

We evaluate cuTAMP on a diverse range of TAMP problems of varying difficulty and highlight the benefits of GPU parallelism. By scaling the number of particles, we achieve significant improvements in the number of satisfying solutions, algorithm runtime, and solution quality. For highly constrained problems that baselines fail to solve, cuTAMP finds solutions in just seconds. We deploy our algorithm on a real UR5 and Kinova arm and showcase its fast planning capabilities for long-horizon manipulation problems (Figures 3 and 4). Code and videos are available on our website: cutamp.github.io.

II. RELATED WORK

Parallelized Motion Planning. Early algorithms for parallelized motion planning used multiprocessing [12] during primarily embarrassingly parallelizable operations, for example, when computing an explicit representation of the robot’s configuration space [13]. More recent algorithms leverage vectorization [14] or GPU-acceleration [15, 16] to implement primitive motion planning operations, such as forward kinematics and collision checking. Our work is most closely related to cuRobo [17], which leverages GPU-acceleration in two phases: first, during a sampling-based Probabilistic Roadmap

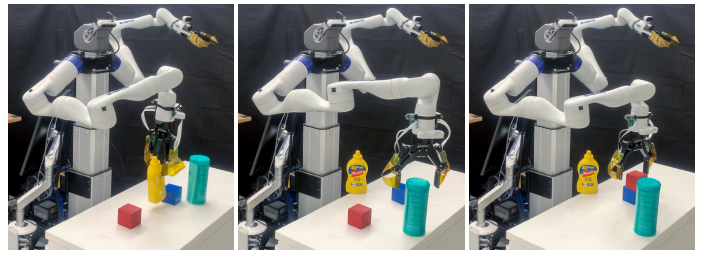


Fig. 4: **Block Stacking with a Kinova Arm.** The objective is to stack the red block on the blue block. However, the mustard bottle and canister obstruct all placements. cuTAMP reasons to move these objects out of the way before placing the red block.

(PRM) [18] phase that generates candidate paths, and second, during a trajectory optimization phase seeded from these paths that minimizes trajectory duration subject to dynamical limits. In contrast, cuTAMP addresses the broader problem of TAMP (i.e., manipulation planning), which requires reasoning about grasps, contacts, and placements in addition to motions.

Sampling-Based TAMP. Sampling-based TAMP algorithms handle the continuous decision-making within TAMP through discretization and composition. They generate values that satisfy specific constraints, such as grasp and placement stability constraints, and intersect them with additional constraints, such as kinematic and collision constraints, through rejection and conditional sampling [1]. Prior approaches do this by sampling a fixed problem discretization [19, 20], combining generators using a custom interface layer [6], searching through the multi-modal continuous space [21, 8], specifying geometric suggesters [5], and composing samplers using a stream specification [7, 2, 22, 23]. Additionally, it is possible to cast satisfaction as an inference problem and leverage techniques like Markov chain Monte Carlo (MCMC) [24] and Stein Variational Inference [25]. We leverage sampling to populate candidate *particles* that are near the solution manifold, but not necessarily feasible, for gradient-based optimization.

Optimization-Based TAMP. In contrast to sampling-based algorithms, which leverage constraint compositionality, optimization-based TAMP algorithms directly solve for continuous parameters that jointly satisfy all plan constraints. Although sampling-based methods can generally optimize plan cost in an anytime mode using rejection sampling, optimization-based TAMP leverages mathematical programming, often via first- and higher-order optimization. Several approaches use off-the-shelf Mixed Integer Programming (MIP) solvers for constraint satisfaction; however, these approaches are limited to simplified TAMP problems where the continuous dynamics are linear [26, 27] or convex [28, 29, 30, 31]. Other approaches directly address the non-convex constrained optimization problem using Sequential Quadratic Programming (SQP) [32], Augmented Lagrangian methods [33, 34, 35, 26, 36, 37], and Alternating Direction Method of Multipliers (ADMM) [38]. These methods are computationally expensive per attempt and are not guaranteed

```

MoveFree( $q_1, q_2 : \text{conf}, \tau : \text{traj}$ )
  con: [Motion( $q_1, \tau, q_2$ ), CFreeTraj( $\tau$ )]
  pre: [AtConf( $q_1$ ), HandEmpty()]
  eff: [AtConf( $q_2$ ),  $\neg$ AtConf( $q_1$ )]
  cost: [TrajLength( $\tau$ )]

Pick( $o : \text{obj}, g : \text{grasp}, p : \text{placement}, q : \text{conf}$ )
  con: [Kin( $q, o, g, p$ ), Grasp( $o, g$ ), CFreeHold( $o, g, q$ )]
  pre: [HandEmpty(), AtConf( $q$ ), AtPlacement( $o, p$ )]
  eff: [Holding( $o, g$ ),  $\neg$ HandEmpty(),  $\neg$ AtPlacement( $o, p$ )]

MoveHold( $o : \text{obj}, g : \text{grasp}, q_1, q_2 : \text{conf}, \tau : \text{traj}$ )
  con: [Motion( $q_1, \tau, q_2$ ), CFreeTrajHold( $o, g, \tau$ )]
  pre: [AtConf( $q_1$ ), Holding( $o, g$ )]
  eff: [AtConf( $q_2$ ),  $\neg$ AtConf( $q_1$ )]
  cost: [TrajLength( $\tau$ )]

Place( $o : \text{obj}, g : \text{grasp}, p : \text{placement}, s : \text{surface}, q : \text{conf}$ )
  con: [Kin( $q, o, g, p$ ), StablePlace( $o, p, s$ ), CFreePlace( $o, p$ )]
  pre: [Holding( $o, g$ ), AtConf( $q$ )]
  eff: [HandEmpty(), AtPlacement( $o, p$ ), On( $o, s$ ),  $\neg$ Holding( $o, g$ )]

```

Listing 1: **Parametrized actions for pick and place tasks.** We list the most important constraints for simplicity of presentation. CFree is an abbreviation for Collision Free.

to converge to a feasible solution. By optimizing thousands of candidates solutions in parallel, our approach is more likely to produce at least one feasible solution.

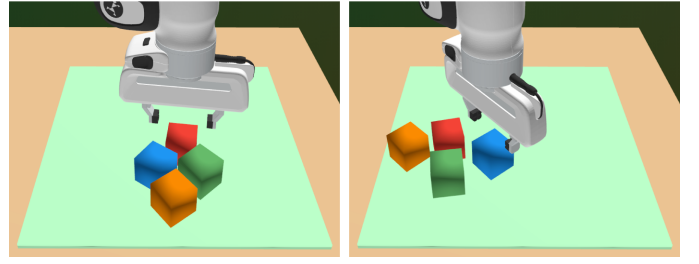
III. PROBLEM FORMULATION

Our approach is generally applicable to long-horizon decision-making problems with both discrete and continuous parameters, such as assembly line design, smart power grid management, and programming video game non-playable characters. We focus on solving TAMP problems.

Let a TAMP problem be a tuple $\Pi = \langle \mathcal{A}, s_0, S_* \rangle$, where $\mathcal{A}$ is a set of *parametrized actions*, s_0 is the initial state, and S_* is the set of goal states. We represent states and actions using a PDDL-style (Planning Domain Definition Language) [39] action language, where states are comprised of Boolean variables corresponding to logical propositions. For example, the $\text{AtPlacement}(o, p)$ *predicate* holds if object o is currently at placement pose p . Each parametrized action $a \in \mathcal{A}$ accepts parameters $\mathbf{x}_a = (x_1, \dots, x_n)$, which may include both discrete and continuous values, and consists of:

- Constraints (**con**) on its parameters, which must all be satisfied in order for the action to be valid in some state. We assume that the constraints are equality or inequality constraints on a differentiable real-valued function, denoted as J_c for each constraint $c \in \text{con}(a)$ in action a .
- Preconditions (**pre**), which must all be true for the action to be executed in a given state.
- Effects (**eff**), which describe propositions that become true or false after executing the action.
- Costs (**cost**) on the parameters, which we aim to reduce.

For example, consider the `Pick` action in Listing 1 for grasping object o , with the grasp g , when at pose p , and corresponding robot configuration q . Its preconditions are 1) `HandEmpty()` – the robot’s hand must be empty, 2) `AtConf( $q$ )` – the robot must be at configuration q , and 3) `AtPlacement( $o, p$ )` – object o must be at placement pose



(a) cuTAMP with goal costs (b) Parallelized Sampling

Fig. 5: **Minimizing Distance between Objects.** The state after executing the best particle. (a) cuTAMP achieves significantly lower cost compared to (b) parallelized sampling.

p . As a result of executing the `Pick` action, `AtGrasp( $o, g$ )` – the robot holds object o with grasp g – is now true, but `HandEmpty()` and `AtPlacement( $o, p$ )` are now false. To execute the action, we require that the kinematic constraint `Kin( $q, o, g, p$ )` is satisfied (i.e., $\text{FK}(q) = p \cdot g$ where FK is forward kinematics), g is a valid grasp for object o as required by `Grasp( $o, g$ )`, and the grasp is Collision-Free (CFree) at configuration q required by `CFreeHold( $o, g, q$ )`. We provide additional `MoveFree`, `MoveHold`, and `Place` actions in Listing 1. The complete description of the constraints and actions considered may be found in Appendix A1.

The objective of the TAMP system is to find a plan skeleton $\pi = (a_1, \dots, a_n)$, a sequence of actions, with valid parameter assignments $\{\mathbf{x}_a \mid a \in \pi\}$ such that when applied from the initial state s_0 , it produces a goal state $s \in S_*$ while minimizing the overall cost.

Running Example. Consider the following skeleton for placing object `red` on surface `table`, where constant continuous parameters are bolded:

$$\pi = [\text{MoveFree}(\mathbf{q}_0, q_1, \tau_1), \text{Pick}(\text{red}, g, \mathbf{p}_0, q_1), \text{MoveHold}(\text{red}, g, q_1, q_2, \tau_2), \text{Place}(\text{red}, g, \mathbf{p}_1, \text{table}, q_2)].$$

Parameters in a plan skeleton may be shared across actions. In order to `Pick` object `red` at configuration q_1 , the robot must first use `MoveFree` to move from its initial configuration $\mathbf{q}_0$ to q_1 . A key challenge in TAMP is that continuous constraints may restrict the set of viable plan skeletons. For example, a blue object may initially obstruct the red object, causing `CFreeHold` in `Pick` to be false. This skeleton would admit no solutions as long as blue is at its initial placement.

Goal Costs. In some tasks, the objective is to reach a state that additionally minimizes a cost function, such as the goal distance between objects (Figure 5). We support this by treating costs on the goal state as a dummy action with costs that are appended as the final action in candidate plan skeletons. In our example, this action for three objects is:

```

MinimizeObjDist( $o_1, o_2, o_3 : \text{obj}, p_1, p_2, p_3 : \text{placement}$ )
  pre: [AtPlacement( $o_1, p_1$ ), AtPlacement( $o_2, p_2$ ),
        AtPlacement( $o_3, p_3$ )]
  cost: [Dist( $p_1, p_2$ ), Dist( $p_2, p_3$ ), Dist( $p_1, p_3$ )]

```

IV. CUTAMP OVERVIEW

cuTAMP is a *sequence-then-satisfy* approach to TAMP [1]: it first searches over plan skeletons and then searches over continuous action parameter values for each of the actions within that skeleton. Specifically, cuTAMP first generates a candidate skeleton and then uses massively parallelized differentiable optimization to solve the induced constraint satisfaction problem (CSP) (Section V). Since any particular skeleton might induce a CSP with no feasible solution, cuTAMP *backtracks* to attempt different skeletons until it finds a feasible one. To ensure efficient optimization, cuTAMP invokes sampling methods to initialize a batch of *particles*, representing candidate *continuous* solutions to the CSP (Section VI). Finally, to minimize backtracking, cuTAMP derives a plan feasibility heuristic to guide the discrete search over skeletons (Section VII). In Section A2, we prove that cuTAMP is *probabilistically complete*.

The cuTAMP algorithm is listed in Algorithm 1. In Stage 1, it searches for N_s initial plan skeletons using task planning, which we perform using a forward best-first search that introduces new parameters for continuous variables whenever it expands a node in the search tree. For each skeleton π_i , it performs particle initialization to get a batch of particles $\mathcal{P}_i$ (Sec. VI), and computes the PLANHEURISTIC (Sec. VII). These candidate skeletons and particles are appended to a priority queue Q , which is ordered by the heuristic values. In Stage 2, it iteratively selects plan skeletons and their particles from Q and optimizes the particles using differentiable optimization (Sec. V-B) until a stopping criterion is met (e.g., time limit or number of steps). After optimizing the particles, it evaluates whether the overall stopping conditions have been met, such as finding at least one satisfying solution or reaching a user-defined cost value. If the conditions are met, it returns all satisfying solutions. Otherwise, it recomputes the plan heuristic on the optimized particles and samples additional skeletons and particles to add to the priority queue.

In the sections that follow, we discuss each component of cuTAMP in detail. At the core of our approach is our use of parallelized differentiable optimization for solving CSPs. We thus present this first in Section V. Like most optimization-based approaches, ours is sensitive to initialization. cuTAMP addresses this challenge with particle initialization strategies, which we present next in Section VI. Finally, we describe our approach for guiding the discrete search over plan skeletons, which leverages the particle initializations, in Section VII.

V. PARALLELIZED CONSTRAINT SATISFACTION

A candidate plan skeleton $\pi = (a_1, \dots, a_n)$ induces a continuous Constraint Satisfaction Problem (CSP), where the goal is to assign values to the continuous parameters such that all the constraints across $a \in \pi$, are satisfied. We denote the set of constraints across π as $\mathbf{con}(\pi) = \bigcup_{a \in \pi} \mathbf{con}(a)$, and the cost functions as $\mathbf{cost}(\pi) = \bigcup_{a \in \pi} \mathbf{cost}(a)$.

A CSP can be visualized as a constraint network, where nodes represent variables and constraints, and edges connect variables via the constraints. Figure 6 depicts the constraint

Algorithm 1 cuTAMP Algorithm

Require: TAMP problem Π , N_b particle batch size, N_s number of new plan skeletons

```

1:  $Q \leftarrow []$  ▷ Initialize priority queue
2: for  $i = 1, \dots, N_s$  do ▷ Stage 1: Initialize plans and particles.
3:    $\pi_i \leftarrow \text{SEARCHPLANSKELETON}(\Pi)$ 
4:    $\mathcal{P}_i \leftarrow \text{INITIALIZEPARTICLES}(\pi_i, N_b)$  ▷ Parallelized Sampling
5:    $h_i \leftarrow \text{PLANHEURISTIC}(\pi_i, \mathcal{P}_i)$ 
6:    $\text{PUSH}(Q, (h_i, \pi_i, \mathcal{P}_i))$ 
7: while  $Q \neq []$  do ▷ Stage 2: Loop over skeletons and optimize.
8:    $(h, \pi, \mathcal{P}) \leftarrow \text{POP}(Q)$ 
9:    $\mathcal{P}' \leftarrow \text{OPTIMIZEPARTICLES}(\pi, \mathcal{P})$  ▷ Parallelized Optimization
10:  if  $\text{ISGOALSATISFIED}(\pi, \mathcal{P}')$  then
11:    return  $\pi$ ,  $\text{GETSATISFYINGPARTICLES}(\pi, \mathcal{P}')$ 
12:  for  $i = 1, \dots, N_s$  do ▷ Sample additional plan skeletons.
13:     $\pi_{\text{new}} \leftarrow \text{SEARCHPLANSKELETON}(\Pi)$ 
14:     $\mathcal{P}_{\text{new}} \leftarrow \text{INITIALIZEPARTICLES}(\pi_{\text{new}}, N_b)$ 
15:     $h_{\text{new}} \leftarrow \text{PLANHEURISTIC}(\pi_{\text{new}}, \mathcal{P}_{\text{new}})$ 
16:     $\text{PUSH}(Q, (h_{\text{new}}, \pi_{\text{new}}, \mathcal{P}_{\text{new}}))$ 
17:   $h' \leftarrow \text{PLANHEURISTIC}(\pi, \mathcal{P}')$ 
18:   $\text{PUSH}(Q, (h', \pi, \mathcal{P}'))$  ▷ Add back to queue
19: return FAILURE

```

network for the running example (Sec. III) for placing object **red** on surface **table**, where the free variables are $q_1, \tau_1, g, q_2, \tau_2$, and p_1 , while the set of constraints $\mathbf{con}(\pi)$ are:

$\{\text{Motion}(\mathbf{q}_0, \tau_1, q_1), \text{CFreeTraj}(\tau_1), \text{Kin}(q_1, \text{red}, g, \mathbf{p}_0),$
 $\text{Grasp}(\text{red}, g), \text{CFreeHold}(\text{red}, g, q_1), \text{Motion}(q_1, \tau_2, q_2),$
 $\text{CFreeTrajHold}(\text{red}, g, \tau_2), \text{Kin}(q_2, \text{red}, g, p_1),$
 $\text{StablePlace}(\text{red}, p_1, \text{table})\}, \text{CFreePlace}(\text{red}, p_1)\}.$

A. Constraint Satisfaction via Optimization

We solve the CSP induced by skeleton π by reducing it to an unconstrained optimization problem involving the real-valued functions comprising each constraint and the cost functions across π . Let a parameter *particle* $\mathbf{x} = (x_1, x_2, \dots, x_{N_v})$ be an assignment to the N_v continuous variables in π . For each constraint $c \in \mathbf{con}(\pi)$, we denote its differentiable real-valued function as J_c and tolerance as ϵ_c . A constraint c is satisfied if $J_c(\mathbf{x}_{\text{param}(c)}) \leq \epsilon_c$, where $\mathbf{x}_{\text{param}(c)}$ denotes the subset of parameters in $\mathbf{x}$ that are relevant to c . In our running example, the kinematics constraint $\text{Kin}(q_1, \text{red}, g, \mathbf{p}_0)$ performs forward kinematics on the robot configuration q_1 and returns the pose error relative to the target pose $\mathbf{p}_0 \cdot g$ for object **red**. A particle $\mathbf{x}$ is *satisfying* if it satisfies all the constraints in π , and hence forms a valid solution to the CSP. Finding such a satisfying particle corresponds to solving the following mathematical program:

$$\begin{aligned}
& \min_{\mathbf{x}} \quad \sum_{c' \in \mathbf{cost}(\pi)} c'(\mathbf{x}_{\text{param}(c')}) \\
& \text{subject to} \quad J_c(\mathbf{x}_{\text{param}(c)}) \leq \epsilon_c \quad \forall c \in \mathbf{con}(\pi).
\end{aligned} \tag{1}$$

Each plan skeleton induces a mathematical program, where the variables and costs are determined by the parameters, constraints, and costs of its actions. We solve the program by relaxing the hard constraints into soft costs, enabling the use of unconstrained optimization. The objective function is a weighted sum of the costs from the hard constraints and the

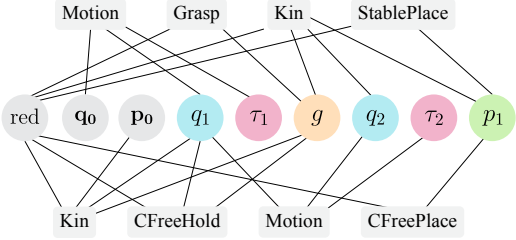


Fig. 6: **Example Constraint Network.** Variables (round nodes) are connected to each other via constraints (rectangular nodes). We omit CFreeTraj constraints for simplicity.

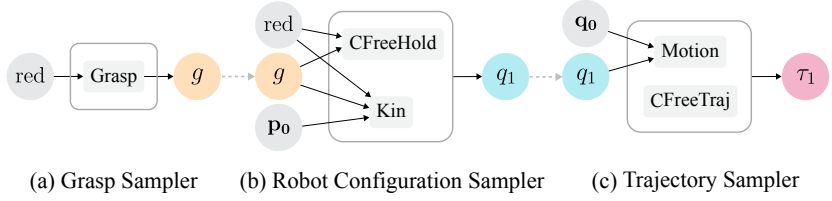


Fig. 7: **Example Sampling Networks.** Samplers solve subgraphs of the constraint network (Fig. 6). Solid arrows indicate the input and output parameters for each sampler. Dotted arrows represent dependencies between the samplers, defining the order in which they can be *composed*.

plan costs, which may be viewed as soft constraints:

$$\min_{\mathbf{x}} \mathcal{J}(\mathbf{x}) = \underbrace{\sum_{c \in \text{con}(\pi)} \lambda_c \cdot J_c(\mathbf{x}_{\text{param}(c)})}_{\text{hard constraint costs}} + \underbrace{\sum_{c' \in \text{cost}(\pi)} \lambda_{c'} \cdot c'(\mathbf{x}_{\text{param}(c')})}_{\text{soft action costs}}, \quad (2)$$

where λ_c is the weight (constant penalty) for the corresponding constraint or cost c , which allows us to balance their influence during the optimization process. We can check whether a particle $\mathbf{x}$ satisfies the CSP by evaluating:

$$\bigwedge_{c \in \text{con}(\pi)} J_c(\mathbf{x}_{\text{param}(c)}) \leq \epsilon_c. \quad (3)$$

B. Parallelized Differentiable Optimization

A key contribution of our work is to exploit parallelism to apply differentiable optimization and explore thousands of parameter particles simultaneously. We first denote a batch of N_b parameter particles as $\mathcal{P} = (\mathbf{x}_1, \dots, \mathbf{x}_{N_b})$. The objective in our unconstrained optimization problem is now to minimize the mean cost over all particles in $\mathcal{P}$:

$$\min_{\mathcal{P}} \mathcal{J}_{\text{batch}}(\mathcal{P}) = \frac{1}{N_b} \sum_{\mathbf{x} \in \mathcal{P}} \mathcal{J}(\mathbf{x}). \quad (4)$$

Gradient-Based Optimization. We solve the unconstrained optimization problem in Equation 4 by iteratively updating the particles $\mathcal{P}$ using a gradient-based optimizer. At each optimization step, we compute the cost function across the batch of particles. Since the cost function is differentiable, we can compute the gradients of the cost with respect to the particles. These gradients are then used by Adam [40], a stochastic first-order optimizer, to update the parameters within each particle. Our results in Section VIII demonstrate that this approach performs remarkably well *even* with simple unconstrained optimization using Adam; however, our approach is compatible with more complex optimizers including augmented Lagrangians [41, 42, 43], coordinate descent [44], and second-order optimizers [45]. We repeat gradient-descent updates until a stopping criterion has been met, such as a maximum optimization time or finding a satisfying particle within the batch. We check whether a particle is satisfying by

comparing whether the costs corresponding to the plan skeleton’s constraints falls within the defined tolerances (Eq. 3).

Parallelizing on GPUs. To efficiently optimize a batch of particles, we avoid the inefficient summation in Eq. 4 by first stacking the assignments of each continuous variable x_i across the batch of N_b particles into a matrix $\mathbf{X}_i$. For example, if $x_i \in \mathbb{R}^7$ represents a 7-DOF robot configuration, then $\mathbf{X}_i \in \mathbb{R}^{N_b \times 7}$. In order to compute the cost in Eq. 2 across batches of particles simultaneously, we implement vectorized versions of the cost functions using PyTorch [46]. These cost functions are differentiable via automatic differentiation, allowing us to compute gradients in parallel. We also leverage differentiable collision checkers and kinematics models within cuRobo [17], which include custom CUDA kernels for both the forward pass and gradient computations. We use the kinematics model to compute pose errors for kinematic constraints. Collision checking approximates the geometry of the robot and movable objects as spheres, enabling massively GPU-accelerated collision checking against static obstacles represented as oriented bounding boxes, meshes, or signed distance fields. We use these collision checkers, which provide informative and smooth gradients, in our cost functions for checking the collision-free and self-collision constraints. See Appendix Listing 2 for example cost functions in Python.

VI. PARTICLE INITIALIZATION

A key desideratum for solving the highly non-convex optimization problem presented above in Section V-B is avoiding local minima. Towards this, cuTAMP implements a novel strategy based on compositional sampling to initialize the particles $\mathcal{P}$ before optimization is run. One common strategy in optimization-based TAMP is to restart with random initializations when stuck in local minima [24]. However, as we show in Section VIII, targeted initialization via conditional sampling skips the early stages of optimization that must move near the solution manifold. This allows us to jump straight to improving the particles according to the constraints jointly.

Let a *parallelized sampler* be a function that takes one or more constraints, possibly involving different constraint types, as input and generates a batch of N_b assignments to the free parameters involved in those constraints. A *conditional parallelized sampler* accepts assignments for some of the free parameters as additional input. A sampler’s objective is to find

assignments that satisfy its constraints; however, this is only possible when the constraints admit satisfying assignments.

Our initialization strategy is to *compose* the generations of multiple samplers, where each sampler solves a subgraph of the constraint network. This provides an initialization for the entire batch of particles $\mathcal{P}$, corresponding to N_b candidate solutions to the CSP. This composition forms a sampling network (also known as a computation graph), an orientation of the edges in a constraint network that transforms it into a directed acyclic graph (DAG) [7].

Concrete Example. Consider again the plan skeleton from our running example in Section III, for which the constraint network is depicted in Figure 6. We outline our use of samplers to initialize particles for the first two actions in the skeleton: 1) `MoveFree`(q_0, q_1, τ_1) and 2) `Pick`(`red`, g, p_0, q_1), where the free variables are q_1, τ_1, g and q_1 . Similar to the approach in [7], we provide the following samplers:

- 1) A 6-DOF grasp sampler that takes the `Grasp`(`red`, g) constraint as input and generates grasps $\mathbf{G}$ in the object frame (Figure 7a).
- 2) A conditional sampler for robot configurations that takes the grasp samples $\mathbf{G}$ and constraints `Kin`(q_1, red, g, p_0) and `CFreeHold`(`red`, g, q_1) as input (Figure 7b). The sampler uses the parallelized inverse kinematics solver from cuRobo [17] to solve for 7-DOF joint positions $\mathbf{Q}_1 \in \mathbb{R}^{N_b \times 7}$ conditioned on the target end-effector poses derived from the grasps $\mathbf{G}$.
- 3) A conditional trajectory sampler for τ_1 that takes the configurations $\mathbf{Q}_1$ and the `Motion`(q_0, τ_1, q_1) and `CFreeTraj`(τ_1) constraints as input (Figure 7c). The sampler could then linearly interpolate between the initial configuration q_0 and $\mathbf{Q}_1$ or sample from a learned diffusion model [47].

By composing these samples, we obtain an initialization for the entire batch of particles $\mathcal{P}$ which corresponds to N_b candidate solutions to the CSP.

Reusing Samples across Skeletons. Constraint subgraphs are often shared across plan skeletons. To avoid duplicated sampling, we cache the outputs of each sampler, allowing us to reuse samples whenever we encounter a shared subgraph. This significantly decreases sampling time by avoiding repeated calls to samplers with the same parameters. In our running example, any plan skeleton involving picking the `red` object at its initial placement p_0 , `Pick`(`red`, g_i, p_0, q_i), shares the subgraph in Figures 7a and 7b, corresponding to the grasp and robot configuration samplers.

VII. APPROXIMATING PLAN SKELETON FEASIBILITY

Now that we have discussed both particle initialization and optimization given a fixed plan skeleton, we turn to the final component of cuTAMP: searching over plan skeletons. A key challenge in sequence-then-satisfy approaches to TAMP is to avoid the *refinement* of plan skeletons with CSPs that are unsolvable, as this results in wasted computational effort. However, proving whether a CSP with non-linear and non-convex constraints is unsolvable is generally intractable. In

sampling-based TAMP, the typical strategy is to sample up to a maximum budget and backtrack upon failure to explore the next skeleton. Our strategy in cuTAMP leverages the thousands of candidate solutions generated during particle initialization to estimate plan skeleton feasibility. We use this feasibility measure, along with other factors such as plan skeleton length, to determine the order in which we refine plan skeletons (PLANHEURISTIC in Algorithm 1).

Recall that our parallelized samplers in Section VI attempt to solve constraint subgraphs (Figure 7). Proving that a subgraph is feasible requires the sampler to find just one counterexample out of the batch of N_b particles. Thus, if any constraints in a plan skeleton have zero satisfying particles, we can say it is likely to be infeasible as we scale N_b .

We use this insight to derive a heuristic for the feasibility of a plan skeleton π with constraints $\mathbf{con}(\pi)$ and particles $\mathcal{P}$:

$$H(\pi, \mathcal{P}) = \frac{1}{|\mathbf{con}(\pi)|} \sum_{c \in \mathbf{con}(\pi)} h(\mathcal{P}, c), \quad (5)$$

$$\text{where } h(\mathcal{P}, c) = \begin{cases} \Lambda_{\text{penalty}} & \text{if } n_{\text{satisfying}} = 0, \\ n_{\text{satisfying}} & \text{otherwise.} \end{cases}$$

$n_{\text{satisfying}} = |J_c(\mathcal{P}_{\text{param}(c)}) \leq \epsilon_c|$ is the number of particles that satisfy constraint c (see Sec. V-A), Λ_{penalty} denotes a large negative penalty applied when no particle in the batch satisfies constraint c . Intuitively, $H(\pi, \mathcal{P})$ measures the average feasibility of constraints across the plan skeleton by counting the number of particles satisfying each constraint, and assigning a large penalty to constraints with no satisfying particles. This heuristic can be quickly evaluated on the GPU.

Pruning Plan Skeletons with Failed Subgraphs. Particle initialization is relatively inexpensive when compared to differentiable optimization, but it still requires non-trivial computation per skeleton. TAMP problems involving many objects may admit hundreds or even thousands of plan skeletons, the majority of which may be infeasible. We seek to detect plan skeletons that have the same pattern of failure as previously unsuccessfully sampled plan skeletons.

To achieve this, we first define a constraint to be likely unsatisfiable if it has zero satisfying particles after sampling. We collect such constraints and prune new skeletons from consideration by the task planner if their constraint networks contain the same unsatisfiable constraint subgraph. For example, in Figure 9, the blue button is out of reach of the Franka, requiring it to use the stick to press the button. The kinematic constraint for pressing the blue button directly without the stick would always have zero satisfying particles. Thus, any plan skeleton that attempts to press the blue button directly can be pruned, as it is infeasible.

Because it is generally intractable to prove constraint unsatisfiability, we periodically re-initialize particles for the parameters involved in the failed constraint subgraphs. If we find a counterexample, i.e., at least one particle that satisfies the constraint, we add all corresponding pruned skeletons back to the search queue, as the constraint has been proven feasible.

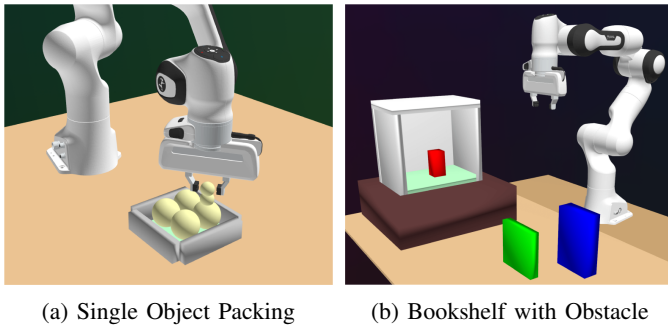


Fig. 8: **TAMP Problems with Obstructions.** (a) Requires packing a square-shaped block. (b) Requires packing the blue and green books into a shelf with a red obstacle.

VIII. EXPERIMENTAL EVALUATION.

We evaluate cuTAMP on a range of simulated TAMP problems with varying levels of difficulty. These problems differ in the number of plan skeletons that are feasible, and the volume of the solution space due to the complex interaction between constraints. We conduct ablation studies on cuTAMP to analyze the impact of particle batch size, tuning of the cost weights λ , and subgraph caching. We consider the following baselines, which emulate existing serial sampling-based [7] and optimization-based [34] planners when run with a single particle ($N_b = 1$):

a) **SAMPLING**: Particles are continuously resampled without optimization via OPTIMIZEPARTICLES.

b) **OPTIMIZATION**: Particles are initialized uniformly within bounds (e.g., joint limits) without sampling.

Experimental Setup. For each approach, we run at least 10 trials and report the coverage (i.e., success rate) along with the mean and 95% confidence interval across metrics including the number of satisfying particles and runtime to find a solution. Grasps are parametrized as top-down 4-DOF or 6-DOF poses, while placements are only 4-DOF poses. Robot configurations correspond to 7-DOF joint positions. Grasps are always sampled and fixed, while placement poses and robot configurations are optimized. To evaluate the generality of cuTAMP, we use the same cost weights across all problems and the same learning rates for Adam. The primary hyperparameter we vary is the particle batch size N_b , in order to investigate how increased parallelism affects performance.

We defer motion generation until after solving for placements and robot configurations. While cuTAMP supports directly jointly optimizing collision-free trajectories (Figure 10), we found that in practice it is more computationally efficient to optimize for collision-free start and end configurations and defer full motion planning until after optimization. Specifically, we iterate through satisfying particles until we find one that admits full motions in a semi-hierarchical fashion. We use the GPU-based motion planner within cuRobo [17]. The timing information we present does not include this motion planning time, which requires only around a few hundred milliseconds for each trajectory parameter and almost always succeeds for our distribution of problems. Additional experimental details

Approach	N_b	Coverage	#Satisfying	Sol. Time (s)
SAMPLING	1	30/30	11.33 ± 1.35	0.648 ± 0.171
	64	30/30	594.70 ± 9.82	0.096 ± 0.011
	256	30/30	1419.20 ± 12.97	0.100 ± 0.002
	1024	30/30	1839.07 ± 14.45	0.163 ± 0.001
OPTIMIZATION	1	4/30	1.00 ± 0.00	3.096 ± 1.914
	64	30/30	5.63 ± 0.95	2.383 ± 0.192
	256	30/30	22.20 ± 1.82	1.857 ± 0.113
	1024	30/30	91.17 ± 3.93	1.394 ± 0.082
cuTAMP	1	26/30	0.85 ± 0.15	0.530 ± 0.101
	64	30/30	46.67 ± 1.37	0.138 ± 0.041
	256	30/30	192.67 ± 2.31	0.099 ± 0.002
	1024	30/30	770.97 ± 5.99	0.163 ± 0.002

TABLE I: **Results on Single Object Packing.** We ablate the particle batch size N_b , where $N_b = 1$ is representative of serial approaches [7, 34]. The #Satisfying metric measures the number of satisfying particles. The best solution time for each approach is bolded, and the overall best is **highlighted**.

and full results can be found in Appendix A4.

A. Solving TAMP Problems with Obstruction

Single Object Packing (Figure 8a). The TAMP planner must find placement poses for the object that do not collide with the grey walls. Each method is given a 5 second budget for resampling or optimization. Table I shows that all methods successfully find solutions in at least some trials. Due to the short horizon of this problem, SAMPLING alone with a small batch size is sufficient to solve it. However, OPTIMIZATION with uniform random initialization performs poorly. Although increasing the batch size improves its performance, it remains $14\times$ slower than the other approaches to find satisfying solutions. This highlights the importance of particle initialization via conditional sampling to start near the solution manifold.

Bookshelf Problem (Figure 8b). The robot can choose either to: 1) pack the books directly into the shelf without removing the red obstacle, or 2) first remove the obstacle before packing the books. The CSP induced by the first strategy is significantly more constrained, requiring increased sampling and optimization time to find a satisfying solution. We sample 6-DOF grasps in this problem, as they are required to ensure collision-free placements of the books.

Our results are presented in Table II. We observe that the plan feasibility heuristic (Eq. 5) prioritizes skeletons that first move the obstacle out of the way, as they result in more satisfying solutions. Both variants of cuTAMP are over 45% faster than SAMPLING to find a solution and achieve full coverage with much smaller batch sizes. Subgraph caching improves the runtime of cuTAMP by 28%, as it speeds up particle initialization over skeletons. However, it can suffer from local minima with smaller batch sizes.

B. Optimizing Goal Costs

We consider minimizing the goal distance between four objects in a goal region (Figure 5). We compare SAMPLING against two versions of cuTAMP: one that explicitly optimizes the goal cost and one without. For each approach in Table III,

Approach	N_b	Coverage	#Opt. Plans	Sol. Time (s)
SAMPLING	64	7/30	1.57 ± 1.05	3.95 ± 3.65
	256	24/30	1.54 ± 0.37	5.83 ± 2.29
	512	28/30	1.25 ± 0.17	7.22 ± 1.72
	2048	30/30	1.27 ± 0.26	11.59 ± 3.15
cuTAMP	64	30/30	1.80 ± 0.65	4.09 ± 1.29
	256	30/30	1.00 ± 0.00	2.11 ± 0.17
	512	30/30	1.03 ± 0.07	2.68 ± 0.19
	2048	30/30	1.00 ± 0.00	6.59 ± 0.72
cuTAMP w. subgraph caching	64	29/30	1.93 ± 0.87	4.61 ± 2.67
	256	30/30	1.03 ± 0.07	1.61 ± 0.39
	512	30/30	1.00 ± 0.00	1.52 ± 0.15
	2048	30/30	1.00 ± 0.00	2.15 ± 0.27

TABLE II: **Results on Bookshelf.** #Opt. Plans metric measures the number of plan skeletons that were optimized or resampled before a solution was found (Stage 2 of Algorithm 1).

Approach	N_b	#Satisfying	Best Cost
SAMPLING	256	3380.70 ± 39.54	63.25 ± 2.19
	512	4249.40 ± 68.54	62.66 ± 2.76
	2048	4795.00 ± 39.19	60.00 ± 3.35
	4096	5136.10 ± 49.83	60.14 ± 3.51
cuTAMP w.o. goal cost	256	122.00 ± 4.10	68.97 ± 4.99
	512	269.40 ± 11.39	63.37 ± 2.48
	2048	1092.80 ± 54.68	61.25 ± 1.58
	4096	1936.20 ± 54.06	58.34 ± 1.86
cuTAMP $\lambda = 0.025$	256	23.90 ± 2.49	44.70 ± 0.55
	512	48.40 ± 4.39	44.57 ± 0.53
	2048	125.60 ± 11.86	45.21 ± 0.65
	4096	331.80 ± 26.76	46.90 ± 0.72

TABLE III: **Optimizing Goal Costs.** We minimize the distance between four objects and compare the best particle cost. λ is the weight applied to the goal cost during optimization.

we allocate 10 seconds for optimization or resampling. Our results demonstrate the clear benefit of combining sampling with differentiable optimization for substantially reducing the goal cost. SAMPLING struggles to find good solutions compared to cuTAMP as shown in Figure 5.

C. Solving Highly-Constrained Problems

The objective in the *Tetris* domain is to pack 5 blocks with non-convex shapes into a tight goal region (Figure 1). This task requires reasoning about precise positions and orientations, as the shapes will only fit if they are arranged in particular configuration modes. Each particle is 90-dimensional (ten 7-DOF arm configurations and five 4-DOF placements), highlighting the complexity of the problem.

Hyperparameter Tuning. We conduct an ablation of cuTAMP where we automatically tune the weight λ_c for each cost c (Eq. 2) using Optuna [48], an off-the-shelf hyperparameter optimization framework, with the number of satisfying particles as the target objective. To reduce the dimensionality of the tuning space, we group costs by type and assign a shared weight to each group (e.g., all kinematic position error terms share the same weight). We tune the cost weights over 220 trials, and provide further details in Appendix A4-D.

Results. We present our results in Table IV. SAMPLING fails

Approach	N_b	Coverage	#Satisfying	Sol. Time (s)
SAMPLING	4096	0/50	–	–
cuTAMP	512	30/50	0.70 ± 0.30	11.51 ± 2.72
	1024	34/50	1.18 ± 0.33	11.55 ± 2.04
	2048	47/50	1.79 ± 0.36	11.82 ± 2.69
	4096	50/50	4.10 ± 0.71	12.21 ± 2.34
cuTAMP Tuned	512	48/50	1.75 ± 0.34	7.13 ± 1.51
	1024	50/50	3.00 ± 0.48	5.38 ± 0.63
	2048	50/50	6.54 ± 0.90	5.63 ± 0.58
	4096	50/50	11.96 ± 1.14	7.99 ± 0.58

TABLE IV: **Results on Tetris 5 blocks.** In cuTAMP Tuned, we tune the cost weights automatically using Optuna [48].

completely, while both tuned and untuned variants of cuTAMP successfully find solutions within seconds. Only 0.3% of the optimized particles are satisfying, underscoring the advantage of sampling and optimizing thousands of candidate solutions in parallel. Larger particle batch sizes N_b consistently improve the success rate, the primary planning metric, especially on highly challenging problems like Tetris. Appendix Figure A.4 shows that the cuTAMP’s runtime remains roughly constant up to a batch size of 512, then scales linearly beyond that.

Automated tuning of the cost weights more than halves the time required to find a solution and nearly triples the number of satisfying particles. The tuned weights generalize to a 3 block variant of Tetris, achieving a similar solve time to untuned cuTAMP while increasing the number of satisfying particles (Table A.4). However, they can perform worse than untuned cuTAMP on other problems due to overfitting (Table A.2).

D. Efficiently Searching over Plan Skeletons

In Stick Button, the robot must press the red, green, and blue buttons using either its fingers or a stick as a tool (Figure 9). Due to the kinematic limitations of the Franka, the robot must use the stick to press the blue and green buttons, as they are out of direct reach, demonstrating cuTAMP’s ability to plan with non-prehensile actions. This results in a large number of plan skeletons that are infeasible or have extraneous actions, such as pressing the blue button with the robot’s fingers or pressing the white distractor buttons. A TAMP planner must quickly identify skeletons likely to be infeasible, and reason that the stick must be grasped near one of its ends to press the blue button without colliding with the gray walls.

Our results in Table V indicate that cuTAMP autonomously infers that the stick must be used, leveraging the plan feasibility heuristic to avoid optimizing plan skeletons that are likely infeasible. This heuristic becomes more accurate as we increase the batch size. While SAMPLING reliably solves stick button with 512 particles, it is over $3\times$ slower than cuTAMP with subgraph caching. Subgraph caching substantially speeds up particle initialization and enables quicker transition to the optimization phase of cuTAMP (Stage 2 in Algorithm 1). In contrast to the Franka, the UR5 can reach all the buttons directly without using the stick (Figure 9). When deployed on the UR5, cuTAMP automatically and implicitly infers this difference in reachability.

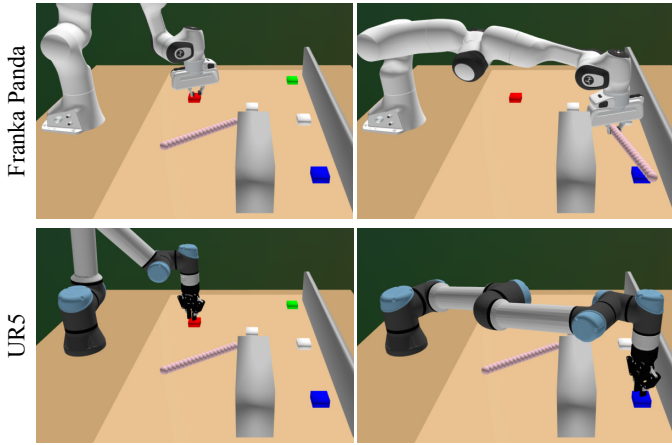


Fig. 9: **Cross-Embodiment Generalization in Stick Button.** The blue button is beyond the reach of the Franka, requiring it to use the stick as a tool. It plans to grasp near the stick’s end to ensure it can press the blue button without colliding into the walls. In contrast, the UR5 can directly reach the button.

Approach	N_b	Coverage	#Opt. Plans	Sol. Time (s)
SAMPLING	32	3/10	2.33 ± 2.87	5.10 ± 2.97
	64	5/10	2.20 ± 2.69	5.15 ± 5.74
	512	10/10	1.00 ± 0.00	4.78 ± 1.16
	2048	10/10	1.00 ± 0.00	12.20 ± 3.77
cuTAMP	32	10/10	4.60 ± 6.44	4.50 ± 2.31
	64	10/10	1.20 ± 0.30	2.94 ± 0.56
	512	10/10	1.00 ± 0.00	4.37 ± 0.55
	2048	10/10	1.00 ± 0.00	10.82 ± 2.41
cuTAMP w. subgraph caching	32	8/10	10.00 ± 10.37	7.96 ± 3.75
	64	10/10	3.30 ± 1.22	8.72 ± 3.33
	512	10/10	1.00 ± 0.00	1.45 ± 0.05
	2048	10/10	1.00 ± 0.00	2.06 ± 0.04

TABLE V: **Results on Stick Button** The #Opt. Plans metric measures the number of plans that were optimized or resampled before a solution was found (Stage 2 of Algorithm 1).

E. Real Robot Experiments

We deploy cuTAMP on two embodiments: 1) a UR5 arm, and 2) a Kinova Gen3 robot. We use the open-world perception strategy from [49] to reconstruct the objects and tabletop. We minimize the goal distance between four objects on a UR5 in Figure 3 and achieve a tight packing. In Figures 4 and 11, cuTAMP autonomously reasons to move obstacles out of the way to achieve the goal on a Kinova Gen3 and UR5, respectively. Finally, we show that cuTAMP supports jointly optimizing grasps, placements, and *full trajectories* parametrized as knot points in Figure 10. Trajectories are initialized by linearly interpolating between the start and end robot configurations. We highly encourage the reader to refer to our [website](#) and supplementary video, which showcases the fast planning capabilities of cuTAMP in the real world.

IX. LIMITATIONS

While cuTAMP supports 6-DOF grasps, we model placement poses with 3-DOF position and yaw (Section VIII). In

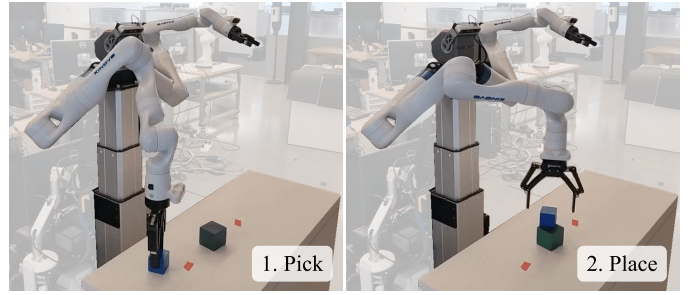


Fig. 10: **Real-World Block Stacking** We jointly optimize grasps, placements, and trajectory knot points using cuTAMP.

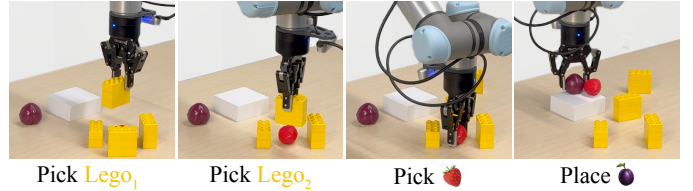


Fig. 11: **Packing fruit with obstacles.** The strawberry is obstructed by four Lego blocks, requiring at least two to be moved for a feasible grasp. cuTAMP autonomously infers this to find a feasible plan skeleton and valid solution.

the future, we would like to also model varying placement roll and pitch, accounting for stable orientations on approximately planar surfaces. Our experiments show that cuTAMP’s optimal configuration is sensitive to the number of particles and cost weights λ , though it performs robustly with the default weights across all tested domains. Future work could explore augmented Lagrangians to dynamically scale costs to satisfy constraints [41, 33], as well as particle pruning to avoid optimizing particles that converge early due to local minima.

cuTAMP leverages parallelism during continuous optimization but does not parallelize the search of plan skeletons. GPU-accelerated task planning [50], via Boolean Satisfiability (SAT) solvers [51], may be able to parallelize all of planning. Our approach addresses deterministic observable planning. Future work involves leveraging GPU acceleration to reason about stochastic actions and partial observability [52, 53]. Extending cuTAMP to tasks involving more complex non-prehensile and contact-rich manipulation is another important direction.

X. CONCLUSION

We propose cuTAMP, the first GPU-parallelized TAMP algorithm for robot manipulators. cuTAMP uses GPU acceleration first during compositional sampling to generate an initial set of plan parameter particles, and second, during joint optimization to solve for particles that satisfy plan constraints and minimize plan costs. We show that increasing the number of particles, particularly beyond a single particle, improves planning runtime and solution quality in challenging, long-horizon manipulation problems. We deploy cuTAMP on several robot embodiments, demonstrating its real-world planning capabilities from perception to action in seconds.

ACKNOWLEDGMENTS

The authors thank Balakumar Sundaralingam for his extensive support with using and debugging cuRobo, and Phillip Isola for providing access to his UR5 robot. We gratefully acknowledge support from NSF grant 2214177; from AFOSR grant FA9550-22-1-0249; from ONR MURI grants N00014-22-1-2740 and N00014-24-1-2603; from the MIT Quest for Intelligence; and from the Robotics and Artificial Intelligence Institute.

REFERENCES

- [1] Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Integrated Task and Motion Planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 2021.
- [2] Caelan R. Garrett, Tomás Lozano-Pérez, and Leslie P. Kaelbling. PDDLStream: Integrating Symbolic Planners and Blackbox Samplers. In *ICAPS*, 2020.
- [3] Jingkai Chen, Jiaoyang Li, Yijiang Huang, Caelan Garrett, Dawei Sun, Chuchu Fan, Andreas Hofmann, Caitlin Mueller, Sven Koenig, and Brian C Williams. Cooperative task and motion planning for multi-arm assembly systems. *arXiv preprint arXiv:2203.02475*, 2022.
- [4] Zhutian Yang, Caelan Reed Garrett, Tomás Lozano-Pérez, Leslie Kaelbling, and Dieter Fox. Sequence-based plan feasibility prediction for efficient task and motion planning. *arXiv preprint arXiv:2211.01576*, 2022.
- [5] Leslie Pack Kaelbling and Tomás Lozano-Pérez. Hierarchical task and motion planning in the now. In *2011 IEEE International Conference on Robotics and Automation*, pages 1470–1477. IEEE, 2011.
- [6] Siddharth Srivastava, Eugene Fang, Lorenzo Riano, Rohan Chitnis, Stuart Russell, and Pieter Abbeel. Combined task and motion planning through an extensible planner-independent interface layer. In *2014 IEEE international conference on robotics and automation (ICRA)*, pages 639–646. IEEE, 2014.
- [7] Caelan Reed Garrett, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Sampling-based methods for factored task and motion planning. *IJRR*, 37(13-14), 2018.
- [8] William Vega-Brown and Nicholas Roy. Asymptotically optimal planning under piecewise-analytic constraints. In *Algorithmic Foundations of Robotics XII: Proceedings of the Twelfth Workshop on the Algorithmic Foundations of Robotics*, pages 528–543. Springer, 2020.
- [9] Xiaolin Fang, Caelan Reed Garrett, Clemens Eppner, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Dieter Fox. Dimsam: Diffusion models as samplers for task and motion planning under partial observability. *arXiv preprint arXiv:2306.13196*, 2023.
- [10] Zhutian Yang, Jiayuan Mao, Yilun Du, Jiajun Wu, Joshua B. Tenenbaum, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Compositional Diffusion-Based Continuous Constraint Solvers. In *CoRL*, 2023.
- [11] Zhigen Zhao, Shuo Chen, Yan Ding, Ziyi Zhou, Shiqi Zhang, Danfei Xu, and Ye Zhao. A survey of optimization-based task and motion planning: From classical to learning approaches. *arXiv preprint arXiv:2404.02817*, 2024.
- [12] Dominik Henrich. Fast motion planning by parallel processing—a review. *Journal of Intelligent and Robotic Systems*, 20:45–69, 1997.
- [13] T. Lozano-Perez and P.A. O’Donnell. Parallel robot motion planning. In *Proceedings. 1991 IEEE International Conference on Robotics and Automation*, pages 1000–1007 vol.2, 1991. doi: 10.1109/ROBOT.1991.131722.
- [14] Wil Thomason, Zachary Kingston, and Lydia E Kavraki. Motions in microseconds via vectorized sampling-based planning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8749–8756. IEEE, 2024.
- [15] Joseph T Kider, Mark Henderson, Maxim Likhachev, and Alla Safonova. High-dimensional planning on the gpu. In *2010 IEEE International Conference on Robotics and Automation*, pages 2515–2522. IEEE, 2010.
- [16] Jia Pan and Dinesh Manocha. Gpu-based parallel collision detection for fast motion planning. *The International Journal of Robotics Research*, 31(2):187–200, 2012.
- [17] Balakumar Sundaralingam, Siva Kumar Sastry Hari, Adam Fishman, Caelan Garrett, Karl Van Wyk, Valts Blukis, Alexander Millane, Helen Oleynikova, Ankur Handa, Fabio Ramos, et al. Curobo: Parallelized collision-free robot motion generation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8112–8119. IEEE, 2023.
- [18] Lydia E Kavraki, Petr Svestka, J-C Latombe, and Mark H Overmars. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. *IEEE transactions on Robotics and Automation*, 12(4):566–580, 1996.
- [19] Caelan Reed Garrett, Tomas Lozano-Perez, and Leslie Pack Kaelbling. Ffrob: Leveraging symbolic planning for efficient task and motion planning. *The International Journal of Robotics Research*, 37(1): 104–136, 2018.
- [20] Neil T Dantam, Zachary K Kingston, Swarat Chaudhuri, and Lydia E Kavraki. An incremental constraint-based framework for task and motion planning. *IJRR*, 37(10), 2018.
- [21] Kris Hauser and Victor Ng-Thow-Hing. Randomized multi-modal motion planning for a humanoid robot manipulation task. *The International Journal of Robotics Research*, 30(6):678–698, 2011.
- [22] Mohamed Khodeir, Ben Agro, and Florian Shkurti. Learning to search in task and motion planning with streams. *IEEE Robotics and Automation Letters*, 8(4): 1983–1990, 2023.
- [23] Tianyu Ren, Georgia Chalvatzaki, and Jan Peters. Extended tree search for robot task and motion planning. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12048–12055. IEEE,

- 2024.
- [24] Marc Toussaint, Cornelius V Braun, and Joaquim Ortiz-Haro. Nlp sampling: Combining mcmc and nlp methods for diverse constrained sampling. *arXiv preprint arXiv:2407.03035*, 2024.
 - [25] Yewon Lee, Philip Huang, Krishna Murthy Jatavallabhula, Andrew Z Li, Fabian Damken, Eric Heiden, Kevin Smith, Derek Nowrouzezahrai, Fabio Ramos, and Florian Shkurti. Stamp: Differentiable task and motion planning via stein variational gradient descent. *arXiv preprint arXiv:2310.01775*, 2023.
 - [26] Caelan Reed Garrett. *Sampling-Based Robot Task and Motion Planning in the Real World*. PhD thesis, Massachusetts Institute of Technology, 2021.
 - [27] Carlos Quintero-Pena, Zachary Kingston, Tianyang Pan, Rahul Shome, Anastasios Kyrillidis, and Lydia E Kavraki. Optimal grasps and placements for task and motion planning in clutter. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3707–3713. IEEE, 2023.
 - [28] Yasser Shoukry, Pierluigi Nuzzo, Alberto L Sangiovanni-Vincentelli, Sanjit A Seshia, George J Pappas, and Paulo Tabuada. Smc: Satisfiability modulo convex optimization. In *Proceedings of the 20th international conference on hybrid systems: Computation and control*, pages 19–28, 2017.
 - [29] Yasser Shoukry, Pierluigi Nuzzo, Alberto L Sangiovanni-Vincentelli, Sanjit A Seshia, George J Pappas, and Paulo Tabuada. Smc: Satisfiability modulo convex programming. *Proceedings of the IEEE*, 106(9):1655–1679, 2018.
 - [30] Enrique Fernández-González, Brian Williams, and Erez Karpas. ScottyActivity: mixed discrete-continuous planning with convex optimization. *Journal of Artificial Intelligence Research*, 62:579–664, 2018.
 - [31] Alphonsus Adu-Bredu, Nikhil Devraj, and Odest Chadwicke Jenkins. Optimal constrained task planning as mixed integer programming. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12029–12036. IEEE, 2022.
 - [32] Dylan Hadfield-Menell, Christopher Lin, Rohan Chitnis, Stuart Russell, and Pieter Abbeel. Sequential Quadratic Programming for Task Plan Optimization. In *Intelligent Robots and Systems (IROS), 2016 IEEE/RSJ International Conference on*, pages 5040–5047. IEEE, 2016.
 - [33] Marc Toussaint. Newton methods for k-order markov constrained motion problems. *arXiv preprint arXiv:1407.0414*, 2014.
 - [34] Marc Toussaint. Logic-geometric programming: an optimization-based approach to combined task and motion planning. In *IJCAI*, 2015.
 - [35] Marc Toussaint, Kelsey Allen, Kevin Smith, and Joshua Tenenbaum. Differentiable physics and stable modes for tool-use and manipulation planning. In *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, June 2018. doi: 10.15607/RSS.2018.XIV.044.
 - [36] Joaquim Ortiz-Haro, Erez Karpas, Michael Katz, and Marc Toussaint. A conflict-driven interface between symbolic planning and nonlinear constraint solving. *IEEE RA-L*, 7(4), 2022.
 - [37] Joaquim Ortiz-Haro, Erez Karpas, Marc Toussaint, and Michael Katz. Conflict-directed diverse planning for logic-geometric programming. *ICAPS*, 2022.
 - [38] Zhigen Zhao, Ziyi Zhou, Michael Park, and Ye Zhao. Sydebo: Symbolic-decision-embedded bilevel optimization for long-horizon manipulation in dynamic environments. *IEEE Access*, 9:128817–128826, 2021.
 - [39] Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. PDDL: The Planning Domain Definition Language. Technical report, Yale Center for Computational Vision and Control, 1998. URL <https://www.cs.cmu.edu/~mmv/planning/readings/98aips-PDDL.pdf>.
 - [40] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
 - [41] John Platt and Alan Barr. Constrained differential optimization. In *Neural Information Processing Systems*, 1987.
 - [42] Benoit Landry, Zachary Manchester, and Marco Pavone. A differentiable augmented lagrangian method for bilevel nonlinear optimization. *arXiv preprint arXiv:1902.03319*, 2019.
 - [43] Eric Heiden, Christopher E Denniston, David Millard, Fabio Ramos, and Gaurav S Sukhatme. Probabilistic inference of simulation parameters via parallel differentiable simulation. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 3638–3645. IEEE, 2022.
 - [44] Stephen J Wright. Coordinate descent algorithms. *Mathematical programming*, 151(1):3–34, 2015.
 - [45] Dong C Liu and Jorge Nocedal. On the limited memory bfgs method for large scale optimization. *Mathematical programming*, 45(1):503–528, 1989.
 - [46] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
 - [47] Huang Huang, Balakumar Sundaralingam, Arsalan Mousavian, Adithyavairavan Murali, Ken Goldberg, and Dieter Fox. Diffusionseeder: Seeding motion optimization with diffusion for rapid motion planning. *arXiv preprint arXiv:2410.16727*, 2024.
 - [48] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 2623–2631, 2019.
 - [49] Aidan Curtis, Xiaolin Fang, Leslie Pack Kaelbling,

- Tomás Lozano-Pérez, and Caelan Reed Garrett. Long-horizon manipulation of unknown objects via task and motion planning with estimated affordances. In *ICRA*, 2022.
- [50] Alessandro Dal Palù, Agostino Dovier, Andrea Formisano, and Enrico Pontelli. Cud@ sat: Sat solving on gpus. *Journal of Experimental & Theoretical Artificial Intelligence*, 27(3):293–316, 2015.
 - [51] Henry A Kautz, Bart Selman, et al. Planning as satisfiability. In *ECAI*, volume 92, pages 359–363. Citeseer, 1992.
 - [52] Leslie Pack Kaelbling and Tomás Lozano-Pérez. Integrated task and motion planning in belief space. *The International Journal of Robotics Research*, 32(9-10): 1194–1227, 2013.
 - [53] Caelan Reed Garrett, Chris Paxton, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Dieter Fox. Online replanning in belief space for partially observable task and motion problems. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5678–5684. IEEE, 2020.
 - [54] Ashwin Deshpande, Leslie Pack Kaelbling, and Tomas Lozano-Perez. Decidability of semi-holonomic prehensile task and motion planning. In *Algorithmic Foundations of Robotics XII: Proceedings of the Twelfth Workshop on the Algorithmic Foundations of Robotics*, pages 544–559. Springer, 2020.
 - [55] William Vega-Brown and Nicholas Roy. Task and motion planning is pspace-complete. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 10385–10392, 2020.
 - [56] Zachary Kingston, Mark Moll, and Lydia E Kavraki. Sampling-based methods for motion planning with constraints. *Annual review of control, robotics, and autonomous systems*, 1(1):159–185, 2018.
 - [57] Shuhei Watanabe. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance. *arXiv preprint arXiv:2304.11127*, 2023.
 - [58] Nikolaus Hansen. The cma evolution strategy: A tutorial. *arXiv preprint arXiv:1604.00772*, 2016.

APPENDIX

Table of Contents:

- Appendix A1: Expanded problem formulation
- Appendix A2: Theoretical analysis
- Appendix A3: Experimental configuration
- Appendix A4: Expanded experimental results

A1. PROBLEM FORMULATION

We expand on our formulation in Section III by fully defining types, predicates, constraints, and actions used in our formulation.

Types:

- `conf` - a robot configuration.
- `traj` - a robot trajectory comprised of a sequence of configurations.
- `obj` - a manipulable object.
- `grasp` - an object grasp pose.
- `placement` - an object placement pose.
- `surface` - a placement surface.
- `button` - a button that can be pressed.
- `press` - a pose at which we press a button.

Predicates:

- `AtConf( $q$ : conf)` - the robot is currently at configuration q .
- `HandEmpty()` - the robot's hand is currently empty.
- `AtPlacement( $o$ : obj,  $p$ : placement)` - object o is currently at placement pose p .
- `Holding( $o$ : obj,  $g$ : grasp)` - object o is currently grasped with grasp pose g .
- `On( $o$ : obj,  $s$ : surface)` - object o is placed on top of surface s .
- `IsStick( $o$ : obj)` - object o is a stick we can use to push a button.
- `Pressed( $b$ : button)` - button b has been pressed.

Constraints:

- `Motion( $q_1$ : conf,  $\tau$ : traj,  $q_2$ : conf)` - τ is a trajectory that connects configurations q_1 and q_2 , and is within the joint limits of the robot.
- `Kin( $q$ : conf,  $o$ : obj,  $g$ : grasp,  $p$ : placement)` - configuration q satisfies a kinematics constraint with placement pose p when object o is grasped with grasp pose g .
- `Grasp( $o$ : obj,  $g$ : grasp)` - g is a valid grasp pose for object o .
- `StablePlace( $o$ : obj,  $p$ : placement,  $s$ : surface)` - p is a stable placement pose for object o on surface s .
- `CFreeTraj( $\tau$ : traj)` - τ is a collision-free trajectory with respect to the objects in the world and does not cause robot self-collisions.
- `CFreeHold( $o$ : obj,  $g$ : grasp,  $q$ : conf)` - g is a collision-free grasp for object o using robot configuration q .

```
PressButton( $b$ : button,  $p$ : press,  $q$ : conf)
  con: [Kin( $q$ ,  $b$ ,  $p$ ), ValidPress( $b$ ,  $p$ ,  $q$ )]
  pre: [HandEmpty(), AtConf( $q$ )]
  eff: [Pressed( $b$ )]
```

```
PressButtonStick( $b$ : button,  $o$ : obj,  $g$ : grasp,  $p$ :
press,  $q$ : conf)
  con: [Kin( $q$ ,  $o$ ,  $g$ ,  $p$ ), ValidStickPress( $o$ ,  $g$ ,  $p$ ,  $b$ )]
  pre: [IsStick( $o$ ), Holding( $o$ ,  $g$ ), AtConf( $q$ )]
  eff: [Pressed( $b$ )]
```

Fig. A.1: **Additional Parametrized Actions.** We list the additional actions required for the Stick Button domain in Figure 9.

- `CFreeTrajHold( $o$ : obj,  $g$ : grasp,  $\tau$ : traj)` - trajectory τ while holding object o with grasp g is collision-free with respect to the objects in the world and does not cause robot self-collisions.
- `CFreePlace( $o$ : obj,  $p$ : placement)` - placing object o at placement pose p is collision-free.
- `ValidPress( $b$ : button,  $p$ : press,  $q$ : conf)` - p is a valid press pose for button b and would make contact.
- `ValidStickPress( $o$ : obj,  $g$ : grasp,  $p$ : press,  $b$ : button)` - p is a valid press pose for button b when using the stick o which is held with grasp pose g , i.e., any point along the stick would make contact with the button.

Actions:

- `MoveFree( $q_1$ ,  $q_2$ : conf,  $\tau$ : conf)` - move in free space from robot configuration q_1 to q_2 using trajectory τ .
- `Pick( $o$ : obj,  $g$ : grasp,  $p$ : placement,  $q$ : conf)` - pick object o , which is at placement pose p , with grasp g and corresponding robot configuration q .
- `MoveHolding( $o$ : obj,  $g$ : grasp,  $p$ : placement,  $q$ : conf)` - move in free space from robot configuration q_1 to q_2 using trajectory τ while holding object o with grasp g .
- `Place( $o$ : obj,  $g$ : grasp,  $p$ : placement,  $s$ : surface,  $q$ : conf)` - place object o which is being held with grasp g on surface s with placement pose p , and corresponding robot configuration q .
- `PressButton( $b$ : button,  $p$ : press,  $q$ : conf)` - press button b with press pose p at corresponding robot configuration q .
- `PressButtonStick( $b$ : button,  $o$ : obj,  $g$ : grasp,  $p$ : press,  $q$ : conf)` - press button b at press pose p with the stick o , which is being held with grasp g , and robot configuration q .

A2. THEORETICAL ANALYSIS OF cuTAMP

In this section, we prove that a simplified version of cuTAMP is probabilistically complete, which informally means that it will almost surely solve nondegenerate feasible problems. Thus, cuTAMP has comparable theoretical properties to prior sampling- and optimization-based TAMP solvers [19, 20, 7, 8]. Note that TAMP is actually decidable [54, 55]; however,

the decomposition-based algorithms provided to prove decidability are computationally inefficient in practice.

Like in constrained motion planning [56], we make the distinction between equality constraints (e.g. kinematics constraints Kin) and inequality constraints (e.g. collision constraints CFreeTraj). We assume that the real-valued function $J_c(\cdot)$ implicitly defining each equality constraint c is continuously differentiable and its Jacobian has full rank. As a result, the set of satisfying values is a lower-dimensional submanifold of the ambient constraint space, and thus the constraint is dimensionality-reducing. Following prior work [56] and usage in practice, we consider equality constraints satisfied if $\|J_c(\mathbf{x}_{\text{param}(c)})\|_2 < \epsilon$ for a small $\epsilon > 0$, namely values close to this submanifold are deemed satisfying, comprising an open subset of the ambient space.

We begin by defining *robust feasibility*, a property of a problem that holds the problem admits a non-degenerate set of solutions. Let $\mathbf{X}_\pi = \{\mathbf{x} \in \mathbf{X} \mid \forall c \in \text{con}(\pi). J_c(\mathbf{x}_{\text{param}(c)}) \leq \epsilon\}$ be the set of parameter values that satisfy the inequality and equality constraints along plan skeleton π .

Definition 1. A plan skeleton $\pi = (a_1, \dots, a_n)$ with a bounded d -dimensional parameter space $\mathbf{X} \subset \mathbb{R}^d$ is *robustly satisfiable* if the set of satisfying parameter values $\mathbf{X}_\pi \subseteq \mathbf{X}$ has positive measure.

Definition 2. A TAMP problem is $\Pi = \langle \mathcal{A}, s_0, S_* \rangle$ *robustly feasible* if there exists a correct and robustly satisfiable plan skeleton $\pi = (a_1, \dots, a_n) \in \mathcal{A}^\infty$ that starting from initial state s_0 that reaches a goal state $s_* \in S_*$.

Finally, we define *probabilistic completeness* and prove cuTAMP is probabilistically complete.

Definition 3. A TAMP algorithm is *probabilistically complete* if for any robustly feasible problem Π , it returns a correct solution in a finite amount of time with probability one.

Theorem 1. *cuTAMP is probabilistically complete.*

Proof: We analyze a simplified version of cuTAMP that initializes parameter particles independently and randomly with positive probability density across $\mathbf{X}$ and omits the plan skeleton feasibility heuristic (Section VII). cuTAMP performs a backtracking plan-space best-first (e.g., breadth-first) search that enumerates the countable set of plan skeletons (Section IV). For a plan skeleton π that is robustly satisfiable, each random initialization has positive probability of sampling a satisfying particle $\mathbf{x} \in \mathbf{X}_\pi$ because $\mathbf{X}_\pi$ has positive measure and cuTAMP samples with positive probability density over $\mathbf{X}_\pi$. Thus, the probability that cuTAMP produces a satisfying particle when randomly restarting the initialization arbitrarily many times is one. Because cuTAMP continuously revisits and resamples all identified plan skeletons, which can alternatively be viewed as creating a separate thread per plan skeleton, once identified, a robustly satisfiable plan skeleton will be randomly restarted arbitrarily many times and thus a solution will be identified with probability one. ■

A3. EXPERIMENTAL CONFIGURATION

We evaluate on an NVIDIA RTX 5880 GPU with 14,080 CUDA cores and 48 GB of VRAM. For comparison, a RTX 4090 has 16,384 CUDA cores and 24 GB of VRAM. Our current implementation of cuTAMP saturates the CUDA cores with larger batch sizes and uses around 2.5GB and 4.5GB of GPU memory for batch sizes of 1024 and 2048, respectively.

Collision Representation. We use the primitive collision checker from cuRobo [17], which approximates static objects in the world as oriented bounding boxes. The robot and movable objects are approximated using collision spheres.

Cost Weights. We use the same set of cost weights λ across all experiments, apart from where we explicitly tune the weights in Tetris with 5 blocks. All weights for constraints are set to 1.0 except for kinematic rotation error (5.0) and stable placements (2.0).

Constraint Tolerances. As exactly satisfying an equality constraint of 0.0 is hard with numerical optimization, we usually set small tolerances. These are the values we used across all experiments. All satisfying particles satisfy all these tolerances.

- Collisions with objects in the world: 1 millimeter (mm).
- Robot Self-Collisions: 0 penetration distance (meters).
- Kinematics: 5mm translational error, 0.05 radians rotational error.
- Robot Joint Limits: 0.
- Stable Placement: 1 centimeter for checking support on a surface, 1mm for checking object contained within a surface.

Real UR5 Configuration. Grasps are sampled and fixed, while placement poses and robot configurations are optimized. Once either a maximum optimization timeout or a set number of satisfying particles is reached, we generate motions using cuRobo. In the supplementary videos, we interleave the generation and execution of these motions on the UR5. A large number of failures in real-world trials can be attributed to inaccuracies in our perception system and a noisy depth sensor (Intel RealSense D435). This results in objects being either under- or over-approximated.

A4. FULL EXPERIMENTAL RESULTS

A. Single Object Packing

We give a maximum resampling or optimization time of 5 seconds after initialization. The full results are in Table A.1. Since cuTAMP uses the same particle initialization phase as SAMPLING, the results are near indistinguishable outside of noise due to randomness.

B. Bookshelf

We provide cuTAMP an optimization budget of 1000 steps per plan skeleton. For the parallelized SAMPLING baseline, we set a maximum of 50 resampling steps per skeleton. We present the full results in Table A.2.

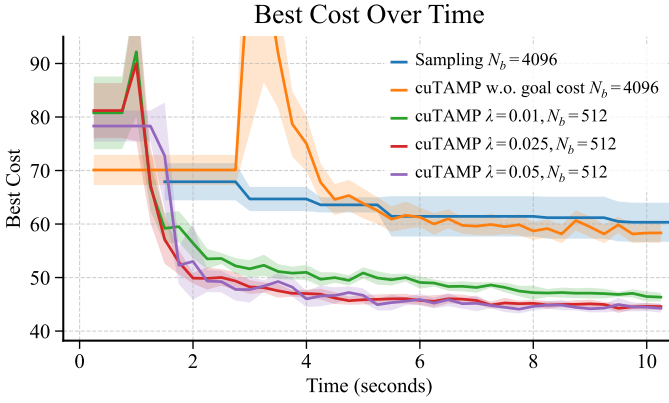


Fig. A.2: **Minimizing Object Distance over Time.** We show the cost of the best particle over time as we resample or optimize the particles. We plot the best batch size for each approach (refer to Table A.3). The early spike in the cost for cuTAMP may be attributed to no warm-up of Adam using a learning rate scheduler, which causes particles to become unsatisfying.

each Optuna trial, we run cuTAMP three times on the Tetris problem using the current set of weights, and use the average number of satisfying particles as the objective to maximize. We first run 20 trials using a Tree-Structured Parzen Estimator (TPE) sampler [57] to efficiently explore the search space and identify promising regions. We then warm-start a Covariance Matrix Adaptation Evolution Strategy (CMA-ES) sampler [58] using the TPE trials, and refine the cost weights for an additional 200 trials.

E. Stick Button

In the SAMPLING baseline, we limit the maximum re-sampling steps per plan skeleton to 50. For cuTAMP, each skeleton undergoes 500 optimization steps. Our full results are presented in Table A.6. Enabling subgraph caching in cuTAMP significantly accelerates the particle initialization over skeletons.

Approach	N_b	#Satisfying	Best Cost
SAMPLING	64	1477.40 ± 24.18	65.54 ± 2.62
	128	2490.80 ± 36.59	65.20 ± 2.70
	256	3380.70 ± 39.54	63.25 ± 2.19
	512	4249.40 ± 68.54	62.66 ± 2.76
	1024	4330.60 ± 79.62	62.49 ± 2.61
	2048	4795.00 ± 39.19	60.00 ± 3.35
	4096	5136.10 ± 49.83	60.14 ± 3.51
cuTAMP w.o. goal cost	64	32.10 ± 3.01	78.86 ± 8.94
	128	61.00 ± 5.10	72.43 ± 6.41
	256	122.00 ± 4.10	68.97 ± 4.99
	512	269.40 ± 11.39	63.37 ± 2.48
	1024	563.60 ± 7.82	61.36 ± 2.95
	2048	1092.80 ± 54.68	61.25 ± 1.58
	4096	1936.20 ± 54.06	58.34 ± 1.86
cuTAMP $\lambda = 0.1$	64	16.40 ± 2.19	47.57 ± 0.75
	128	25.20 ± 3.32	47.62 ± 1.24
	256	57.80 ± 4.84	46.67 ± 0.69
	512	123.40 ± 9.30	46.34 ± 0.81
	1024	233.20 ± 10.66	47.52 ± 0.91
	2048	566.10 ± 23.32	48.60 ± 1.50
	4096	1506.50 ± 35.92	49.53 ± 1.13
cuTAMP $\lambda = 0.25$	64	8.00 ± 2.55	46.60 ± 0.92
	128	13.20 ± 2.15	45.90 ± 0.90
	256	23.90 ± 2.49	44.70 ± 0.55
	512	48.40 ± 4.39	44.57 ± 0.53
	1024	79.90 ± 6.81	45.31 ± 0.80
	2048	125.60 ± 11.86	45.21 ± 0.65
	4096	331.80 ± 26.76	46.90 ± 0.72
cuTAMP $\lambda = 0.5$	64	0.90 ± 0.79	45.87 ± 1.55
	128	1.80 ± 0.74	45.12 ± 0.84
	256	4.40 ± 1.40	45.17 ± 1.98
	512	7.50 ± 1.85	44.26 ± 0.81
	1024	10.60 ± 1.94	44.35 ± 0.70
	2048	10.10 ± 2.22	45.03 ± 1.05
	4096	5.30 ± 1.17	46.51 ± 1.07

TABLE A.3: **Full Results on Minimizing Object Distance.** The goal cost is the sum of the distance between the four objects in centimeters. λ is the cost weight we use for the goal cost in the unconstrained optimization problem for cuTAMP. All approaches achieved full coverage.

Approach	N_b	Coverage	#Satisfying	Sol. Time (s)
SAMPLING	64	0/30	–	–
	128	0/30	–	–
	256	0/30	–	–
	512	0/30	–	–
	1024	1/30	1.00 ± 0.00	12.31 ± 0.00
	2048	1/30	1.00 ± 0.00	18.04 ± 0.00
	4096	0/30	–	–
	8192	4/30	1.00 ± 0.00	48.94 ± 23.99
cuTAMP	64	23/30	1.87 ± 0.63	1.62 ± 0.44
	128	30/30	2.47 ± 0.66	1.57 ± 0.35
	256	29/30	4.93 ± 0.90	1.25 ± 0.22
	512	30/30	8.77 ± 1.22	1.07 ± 0.12
	1024	30/30	17.33 ± 2.00	1.07 ± 0.06
	2048	30/30	36.50 ± 2.80	1.30 ± 0.04
	4096	30/30	76.03 ± 4.17	1.99 ± 0.06
	8192	30/30	157.47 ± 5.92	3.43 ± 0.05
cuTAMP Tuned on 5 Blocks	64	30/30	2.00 ± 0.60	1.78 ± 0.49
	128	29/30	3.59 ± 0.82	1.71 ± 0.35
	256	30/30	7.23 ± 0.88	1.24 ± 0.12
	512	30/30	14.07 ± 1.35	1.09 ± 0.09
	1024	30/30	27.40 ± 2.05	1.16 ± 0.05
	2048	30/30	58.47 ± 3.89	1.36 ± 0.04
	4096	30/30	117.27 ± 6.70	2.05 ± 0.05
	8192	30/30	227.87 ± 12.67	3.54 ± 0.06

TABLE A.4: **Results on Tetris with 3 blocks.** Parallelized SAMPLING only succeeds due to chance of sampling a valid solution in the limit of more samples. The difference in the time to the first solution is minimal between cuTAMP and cuTAMP tuned on Tetris with 5 blocks. However cuTAMP tuned finds 47% more satisfying particles on average.

Approach	N_b	Coverage	#Satisfying	Sol. Time (s)
SAMPLING	64	0/50	–	–
	128	0/50	–	–
	256	0/50	–	–
	512	0/50	–	–
	1024	0/50	–	–
	2048	0/50	–	–
	4096	0/50	–	–
	8192	0/50	–	–
cuTAMP	64	5/50	0.60 ± 0.68	15.46 ± 8.31
	128	6/50	0.67 ± 0.54	13.37 ± 5.82
	256	16/50	0.69 ± 0.32	9.50 ± 3.53
	512	30/50	0.70 ± 0.30	11.51 ± 2.72
	1024	34/50	1.18 ± 0.33	11.55 ± 2.04
	2048	47/50	1.79 ± 0.36	11.82 ± 2.69
	4096	50/50	4.10 ± 0.71	12.21 ± 2.34
	8192	50/50	7.98 ± 0.76	14.07 ± 1.48
cuTAMP Tuned	64	16/50	0.50 ± 0.34	11.53 ± 4.15
	128	36/50	0.61 ± 0.19	11.91 ± 1.77
	256	44/50	1.05 ± 0.26	8.54 ± 1.47
	512	48/50	1.75 ± 0.34	7.13 ± 1.51
	1024	50/50	3.00 ± 0.48	5.38 ± 0.63
	2048	50/50	6.54 ± 0.90	5.63 ± 0.58
	4096	50/50	11.96 ± 1.14	7.99 ± 0.58
	8192	50/50	23.44 ± 1.66	11.29 ± 0.39

TABLE A.5: **Full Results on Tetris with 5 blocks.** Parallelized SAMPLING fails across all batch sizes. While untuned cuTAMP with 256 particles has a low solve time, it has very low coverage. This could be explained by noise in the sampling process, meaning a good initialization has been sampled by chance. Tuned cuTAMP significantly increases the number of satisfying particles and decreases the time required to find a solution.

Approach	N_b	Coverage	#Opt. Plans	Sol. Time (s)
SAMPLING	8	1/10	1.00 ± 0.00	1.68 ± 0.00
	16	4/10	6.75 ± 8.46	13.48 ± 14.73
	32	3/10	2.33 ± 2.87	5.10 ± 2.97
	64	5/10	2.20 ± 2.69	5.15 ± 5.74
	128	8/10	1.50 ± 0.63	4.73 ± 2.00
	256	10/10	1.20 ± 0.30	4.50 ± 1.91
	512	10/10	1.00 ± 0.00	4.78 ± 1.16
	1024	10/10	1.00 ± 0.00	8.54 ± 2.23
	2048	10/10	1.00 ± 0.00	12.20 ± 3.77
	4096	10/10	1.00 ± 0.00	16.97 ± 4.35
cuTAMP	8	5/10	14.40 ± 17.83	13.88 ± 7.29
	16	10/10	2.90 ± 1.63	6.27 ± 2.57
	32	10/10	4.60 ± 6.44	4.50 ± 2.31
	64	10/10	1.20 ± 0.30	2.94 ± 0.56
	128	10/10	1.10 ± 0.23	3.33 ± 0.26
	256	10/10	1.00 ± 0.00	3.28 ± 0.47
	512	10/10	1.00 ± 0.00	4.37 ± 0.55
	1024	10/10	1.00 ± 0.00	6.88 ± 1.13
	2048	10/10	1.00 ± 0.00	10.82 ± 2.41
	4096	10/10	1.00 ± 0.00	21.50 ± 5.40
cuTAMP w. subgraph caching	8	1/10	11.00 ± 0.00	29.05 ± 0.00
	16	6/10	8.67 ± 12.12	10.04 ± 14.77
	32	8/10	10.00 ± 10.37	7.96 ± 3.75
	64	10/10	3.30 ± 1.22	8.72 ± 3.33
	128	10/10	4.20 ± 2.82	11.14 ± 8.14
	256	10/10	1.00 ± 0.00	1.46 ± 0.12
	512	10/10	1.00 ± 0.00	1.45 ± 0.05
	1024	10/10	1.00 ± 0.00	1.75 ± 0.10
	2048	10/10	1.00 ± 0.00	2.06 ± 0.04
	4096	10/10	1.00 ± 0.00	3.17 ± 0.04

TABLE A.6: **Full Results on Stick Button.** We present our results on Stick Button for different batch sizes. SAMPLING with a batch size of 8 happens to find a solution very quickly in 1/10 trials by pure chance.

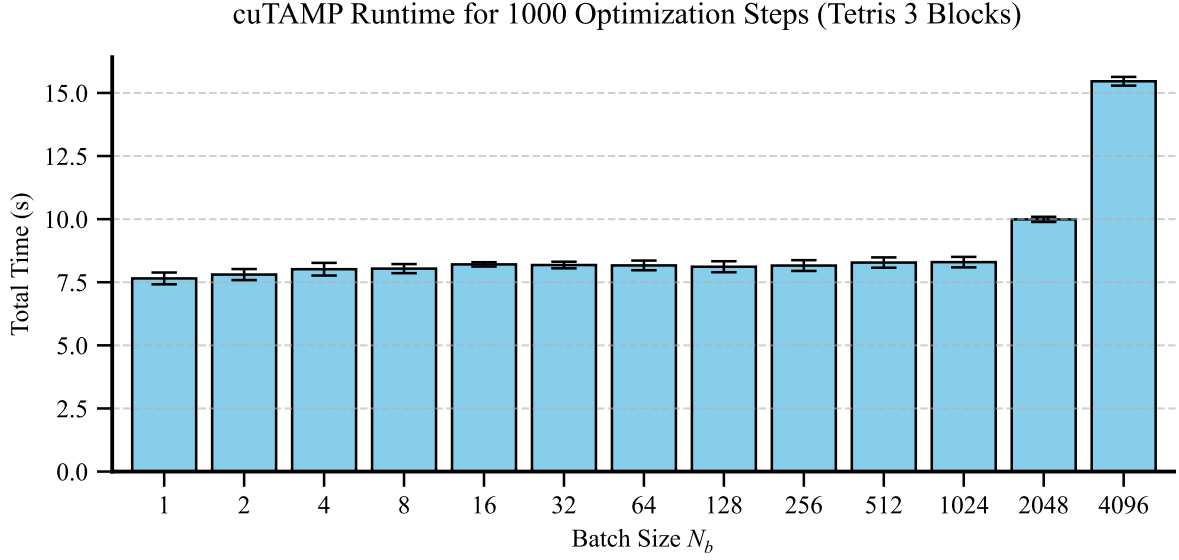


Fig. A.3: **Runtime of cuTAMP in Tetris with 3 Blocks.** We show the runtime behavior of cuTAMP, which includes particle initialization and 1000 differentiable optimization steps, across doubly increasing batch sizes. Results are averaged over 10 trials. The runtime remains nearly constant for up to 1024 particles, beyond which we observe linear scaling.

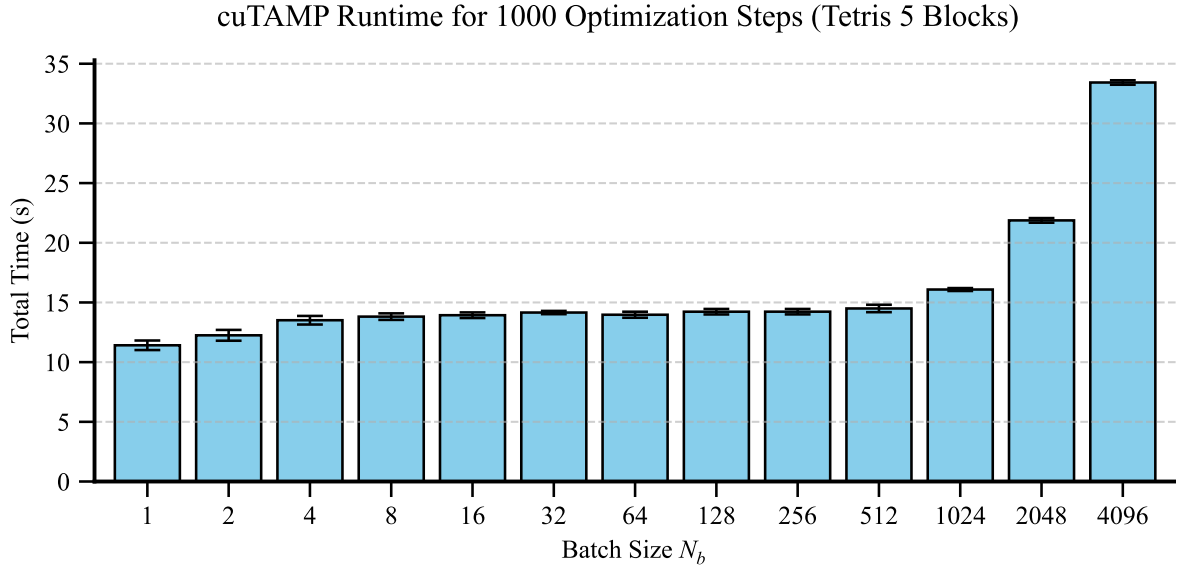


Fig. A.4: **Runtime of cuTAMP in Tetris with 5 Blocks.** We show the runtime behavior of cuTAMP, which includes particle initialization and 1000 differentiable optimization steps, across doubly increasing batch sizes. Results are averaged over 10 trials. The runtime remains nearly constant for up to 512 particles, beyond which we observe linear scaling. This problem is more difficult and involves more constraints than Tetris with 3 blocks (Figure A.3), hence we observe constant scaling only up to a smaller batch size.

```

1  import torch
2  from jaxtyping import Float
3
4  from curobo.types.math import Pose
5
6
7  def curobo_pose_error(
8      pose_a_mat4x4: Float[torch.Tensor, "b *h 4 4"], pose_b_mat4x4: Float[torch.Tensor, "b *h 4 4"]
9  ) -> (Float[torch.Tensor, "b *h"], Float[torch.Tensor, "b *h"]):
10     """
11     Computes the translational and rotational errors between two poses using curobo.
12     Used for computing end-effector pose errors for kinematic constraints.
13     """
14     # Flatten
15     pose_a_flat = pose_a_mat4x4.view(-1, 4, 4)
16     pose_b_flat = pose_b_mat4x4.view(-1, 4, 4)
17
18     # Create curobo pose
19     pose_a = Pose.from_matrix(pose_a_flat)
20     pose_b = Pose.from_matrix(pose_b_flat)
21
22     # Compute distance and unflatten
23     p_dist_flat, quat_dist_flat = pose_a.distance(pose_b)
24     p_dist = p_dist_flat.view(pose_a_mat4x4.shape[:-2])
25     quat_dist = quat_dist_flat.view(pose_b_mat4x4.shape[:-2])
26     return p_dist, quat_dist
27
28
29  def dist_from_bounds(
30      vals: Float[torch.Tensor, "b *h d"],
31      lower: Float[torch.Tensor, "d"],
32      upper: Float[torch.Tensor, "d"],
33  ) -> Float[torch.Tensor, "b *h"]:
34     """
35     Euclidean distance of values from the given lower and upper bounds. If within the bounds, returns 0.
36     Used to check if objects are within the goal region, robot joint limits, etc.
37     """
38     diff_lower = lower - vals
39     diff_upper = vals - upper
40     diff_max = torch.maximum(diff_lower, diff_upper)
41     diff_max = diff_max.clamp(min=0.0)
42     dists = diff_max.norm(p=2, dim=-1)
43     return dists
44
45
46  def obj_dist(goal_obj_position: Float[torch.Tensor, "b n 3"]) -> Float[torch.Tensor, "b"]:
47     """
48     Computes the total distance between all pairs of goal object positions.
49     Can be used to minimize or maximize the distance between objects.
50     """
51     all_obj_dists = torch.cdist(goal_obj_position, goal_obj_position, p=2) # (b, n, n)
52     mask = torch.triu(torch.ones_like(all_obj_dists), diagonal=1) == 1
53     obj_dists = all_obj_dists[mask].view(mask.shape[0], -1) # reshape into num pairs
54     dists_sum = obj_dists.sum(-1)
55     return dists_sum

```

Listing 2: Example Cost Functions. We present vectorized implementations for the following: `curobo_pose_error`: computes the error between batches of poses, used in kinematic costs. `dist_from_bounds`: calculates the distance from boundaries, used for verifying whether an object is within the goal region or whether configurations are within a robot’s joint limits. `obj_dist`: computes the distance between object positions, employed in the soft cost to minimize object distance (see Figures 3 and 5).